

Constrained Optimization Using the Big-M Simplex Method & Transportation Problem (VAM & MODI Method)

Assignment 1 | Computational Report

Report Overview

This report documents the formulation, standard-form conversion, and computational solution of two classical constrained-optimization problems, implemented from scratch in Python (using only NumPy). Both problems are solved iteratively, verified against optimality conditions, and interpreted in the context of the decision variables involved.

01	Big-M Simplex Method	Solving a constrained Linear Programming Problem (LPP) with equality, $\geq$, and $\leq$ constraints
02	Transportation Problem	Obtaining an initial feasible solution using Vogel's Approximation Method (VAM) and testing / confirming optimality using the Modified Distribution (MODI) Method

Technical Summary

Language	Python 3
Libraries Used	NumPy
Methods Implemented	Big-M Simplex, Vogel's Approximation Method (VAM), MODI Method
Solution Approach	Fully iterative / tableau-based, coded manually (no external LP solver)

PART 1 Big-M Simplex Method

1.1 Problem Statement

A constrained Linear Programming Problem (LPP) is selected as the case study. The objective is to determine the values of two decision variables that minimize a linear cost function, subject to one equality constraint, one $\geq$ (greater-than-or-equal-to) constraint, and one $\leq$ (less-than-or-equal-to) constraint.

Minimize:

$$Z = 4x_1 + x_2$$

Subject to:

$$3x_1 + x_2 = 3$$

$$4x_1 + 3x_2 \geq 6$$

$$x_1 + 2x_2 \leq 4$$

$$x_1, x_2 \geq 0$$

1.2 Conversion to Standard Form

Because the model has a mix of '=', ' $\geq$ ' and ' $\leq$ ' constraints, slack, surplus and artificial variables are introduced so that every constraint becomes an equality, which is required by the Simplex tableau. A very large positive penalty **M** is attached to every artificial variable in the objective function so that the Simplex algorithm is forced to drive them out of the optimal basis (the "Big-M" penalty method).

Constraint	Type	Variable(s) Added	Role
$3x_1 + x_2 = 3$	Equality	a1 (artificial)	Provides an initial basic feasible starting point
$4x_1 + 3x_2 \geq 6$	$\geq$	s2 (surplus, -), a2 (artificial)	Surplus removes the excess; artificial restores a unit column
$x_1 + 2x_2 \leq 4$	$\leq$	s3 (slack, +)	Absorbs the unused capacity

The resulting standard-form model (variable order $x_1, x_2, s_1, s_2, a_1, a_2, s_3$) is:

$$3x_1 + x_2 + a_1 = 3$$

$$4x_1 + 3x_2 - s_2 + a_2 = 6$$

$$x_1 + 2x_2 + s_3 = 4$$

$$\text{Minimize } Z = 4x_1 + x_2 + M \cdot a_1 + M \cdot a_2$$

Note: in the code, minimization is carried out by maximizing the negated objective coefficients ($c = [-4, -1, 0, 0, -M, -M, 0]$), and the final answer is negated back ($\text{min_value} = -\text{max_value}$).

1.3 Solution Algorithm

- Start with a_1, a_2 and s_3 as the initial basic variables (basis = $[4, 5, 2]$).
- Compute the reduced cost row: $c - c_B \cdot A$ for every column.
- If the largest reduced cost is ≤ 0 , the current solution is optimal and the loop stops.

- Otherwise, select the entering variable (largest positive reduced cost) and apply the minimum-ratio test on $b_i / A_{i, \text{entering}}$ to select the leaving variable.
- Pivot the tableau (normalize the pivot row, then eliminate the entering column from every other row) and update the basis.
- Repeat until optimality is reached, then read x_1 , x_2 and Z off the final tableau.

1.4 Python Implementation

bigm.py

```
import numpy as np

# Big-M Simplex Method

# Minimize:
# Z = 4x1 + x2

# Subject to:
# 3x1 + x2 = 3
# 4x1 + 3x2 >= 6
# x1 + 2x2 <= 4

M = 1000000.0

# Variable order:
# x1, x2, s1, s2, a1, a2, s3

A = np.array([
    [3, 1, 0, 0, 1, 0, 0],
    [4, 3, 0, -1, 0, 1, 0],
    [1, 2, 1, 0, 0, 0, 1]
], dtype=float)

b = np.array([
    3,
    6,
    4
], dtype=float)

# Convert minimization into maximization

c = np.array([
    -4,
    -1,
    0,
    0,
    -M,
    -M,
    0
], dtype=float)

# Initial basis:
# a1, a2, s3

basis = [4, 5, 2]

while True:

    cb = c[basis]

    reduced_cost = c - cb @ A

    # Check optimality
```

```
if max(reduced_cost) <= 1e-9:
    break

# Entering variable

entering = int(np.argmax(reduced_cost))

ratios = []

for i in range(len(b)):

    if A[i, entering] > 1e-9:

        ratios.append(
            b[i] / A[i, entering]
        )

    else:

        ratios.append(
            float("inf")
        )

# Leaving variable

leaving = int(np.argmin(ratios))

# Pivot operation

pivot = A[leaving, entering]

A[leaving] = A[leaving] / pivot

b[leaving] = b[leaving] / pivot

for i in range(len(b)):

    if i != leaving:

        factor = A[i, entering]

        A[i] = A[i] - factor * A[leaving]

        b[i] = b[i] - factor * b[leaving]

basis[leaving] = entering

# Store solution

solution = np.zeros(7)

for i in range(len(basis)):

    solution[basis[i]] = b[i]
```

```
max_value = c[basis] @ b

min_value = -max_value

print("Optimal x1 =", round(solution[0], 4))

print("Optimal x2 =", round(solution[1], 4))

print("Minimum Z =", round(min_value, 4))
```

1.5 Computational Output

■ Console Output

```
Optimal x1 = 0.4
Optimal x2 = 1.8
Minimum Z = 3.4
```

1.6 Result & Interpretation

Optimal Solution	$x_1 = 0.4$, $x_2 = 1.8$
Minimum Objective Value	$Z = 3.4$

Both artificial variables a_1 and a_2 leave the basis at optimality (their optimal values are 0), confirming that the original three constraints are satisfiable and the solution is feasible with respect to the true (non-penalized) problem. At $x_1 = 0.4$, $x_2 = 1.8$, the equality constraint $3x_1 + x_2 = 3$ is satisfied exactly, the $\geq$ constraint $4x_1 + 3x_2 = 7.0 \geq 6$ holds with slack, and the $\leq$ constraint $x_1 + 2x_2 = 4.0 \leq 4$ is met at its boundary. The minimum achievable cost is therefore $Z = 3.4$.

PART 2 Transportation Problem

2.1 Problem Statement

A classical transportation problem is used as the case study, involving 3 sources (supply points) and 4 destinations (demand points). The objective is to determine a shipment plan that satisfies every source's supply and every destination's demand at the minimum possible total transportation cost. An initial basic feasible solution is first obtained using **Vogel's Approximation Method (VAM)**, and its optimality is then verified using the **Modified Distribution (MODI) Method**.

Cost Matrix, Supply and Demand

Source \ Destination	D1	D2	D3	D4	Supply
S1	4	6	8	6	50
S2	3	5	2	5	60
S3	3	9	6	5	25
Demand	30	40	50	15	135 / 135

Total supply ($50+60+25 = 135$) equals total demand ($30+40+50+15 = 135$), so the problem is balanced and no dummy row/column is required.

2.2 Step 1 — Vogel's Approximation Method (VAM)

VAM builds an initial allocation by repeatedly choosing the row or column with the largest “penalty” (the difference between the lowest and second-lowest cost in that row/column), and allocating as much as possible to the cheapest cell in that row/column. This typically produces a starting solution that is much closer to optimal than simpler methods (e.g. the North-West Corner rule).

- Compute a penalty for every remaining row and column (2nd-lowest cost – lowest cost).
- Pick the row/column with the maximum penalty; allocate to its minimum-cost cell as much as supply/demand allows.
- Reduce the corresponding supply and demand; remove any row/column that is exhausted.
- Repeat until all supply and demand has been allocated.

2.3 Step 2 — MODI (Modified Distribution) Optimality Test

The MODI method tests whether the VAM solution is optimal. Row potentials (u_i) and column potentials (v_j) are computed for every basic (allocated) cell using $u_i + v_j = \text{cost}_{ij}$, fixing $u_1 = 0$ as the reference. For every non-basic (empty) cell, the opportunity cost D_{ij} is computed as $D_{ij} = \text{cost}_{ij} - (u_i + v_j)$. If every D_{ij} is ≥ 0 , the current allocation is already optimal; if any D_{ij} is < 0 , that cell should enter the basis via a stepping-stone loop to further reduce cost.

2.4 Python Implementation

modi.py

```
import numpy as np

cost = np.array([
    [4, 6, 8, 6],
    [3, 5, 2, 5],
    [3, 9, 6, 5]
], dtype=float)

supply = [
    50,
    60,
    25
]

demand = [
    30,
    40,
    50,
    15
]

def vam(cost, supply, demand):

    supply = supply.copy()

    demand = demand.copy()

    m, n = cost.shape

    allocation = np.zeros((m, n))

    rows = set(range(m))

    cols = set(range(n))

    while rows and cols:

        penalties = []

        # Row penalties

        for i in rows:
```

```
        values = sorted(
            cost[i, j] for j in cols
        )

        if len(values) > 1:

            penalty = values[1] - values[0]

        else:

            penalty = values[0]

        penalties.append(
            (penalty, "row", i)
        )

# Column penalties

for j in cols:

    values = sorted(
        cost[i, j] for i in rows
    )

    if len(values) > 1:

        penalty = values[1] - values[0]

    else:

        penalty = values[0]

    penalties.append(
        (penalty, "col", j)
    )

_, kind, index = max(penalties)

if kind == "row":

    i = index

    j = min(
        cols,
        key=lambda x: cost[i, x]
    )

else:
```

```
j = index

i = min(
    rows,
    key=lambda x: cost[x, j]
)

quantity = min(
    supply[i],
    demand[j]
)

allocation[i, j] = quantity

supply[i] -= quantity

demand[j] -= quantity

if supply[i] == 0:
    rows.remove(i)

if demand[j] == 0:
    cols.remove(j)

return allocation

def modi_optimality_test(cost, allocation):

    m, n = cost.shape

    u = [None] * m

    v = [None] * n

    u[0] = 0

    basic = []

    for i in range(m):
```

```
        for j in range(n):

            if allocation[i, j] > 0:

                basic.append(
                    (i, j)
                )

    changed = True

    while changed:

        changed = False

        for i, j in basic:

            if u[i] is not None and v[j] is None:

                v[j] = cost[i, j] - u[i]

                changed = True

            elif v[j] is not None and u[i] is None:

                u[i] = cost[i, j] - v[j]

                changed = True

    delta = np.zeros((m, n))

    for i in range(m):

        for j in range(n):

            delta[i, j] = cost[i, j] - (
                u[i] + v[j]
            )

    return u, v, delta

# VAM Solution

allocation = vam(
```

```
cost,
supply,
demand
)

print(
    "Initial Basic Feasible Solution using VAM:"
)

print(allocation)

initial_cost = np.sum(
    allocation * cost
)

print(
    "\nTransportation Cost =",
    initial_cost
)

# MODI Test

u, v, delta = modi_optimality_test(
    cost,
    allocation
)

print("\nMODI Potentials:")

print("u =", u)

print("v =", v)

print(
    "\nOpportunity Cost Matrix:"
)

print(delta)

if np.all(delta >= -1e-9):

    print(
        "\nAll opportunity costs are non-negative."
    )
```

```
        print(
            "Therefore, the VAM solution is optimal."
        )

    else:

        print(
            "\nNegative opportunity cost found."
        )

        print(
            "Further MODI improvement is required."
        )

    print(
        "\nOptimal Allocation:"
    )

    print(allocation)

    print(
        "\nMinimum Transportation Cost =",
        initial_cost
    )
```

2.5 Computational Output

■ Console Output

Initial Basic Feasible Solution using VAM:

```
[[ 0. 40.  0. 10.]
 [ 5.  0. 50.  5.]
 [25.  0.  0.  0.]]
```

Transportation Cost = 515.0

MODI Potentials:

```
u = [0, -1.0, -1.0]
v = [4.0, 6.0, 3.0, 6.0]
```

Opportunity Cost Matrix:

```
[[0. 0. 5. 0.]
 [0. 0. 0. 0.]
```

[0. 4. 4. 0.]]

All opportunity costs are non-negative.

Therefore, the VAM solution is optimal.

Minimum Transportation Cost = 515.0

2.6 Optimal Shipment Plan

Source \ Destination	D1	D2	D3	D4
S1	—	40	—	10
S2	5	—	50	5
S3	25	—	—	—

2.7 Result & Interpretation

Optimality Check	All Opportunity Costs ≥ 0 (Optimal)
Minimum Transportation Cost	Rs. 515.00

The MODI test shows every opportunity cost in the unallocated cells is non-negative (the smallest values are exactly 0, with others at 4 or 5), so no further reallocation can reduce the cost. The VAM starting solution is therefore already optimal, and the minimum total transportation cost is **515**: S1 ships 40 units to D2 and 10 units to D4; S2 ships 5 units to D1, 50 units to D3 and 5 units to D4; and S3 ships its full supply of 25 units to D1.

Overall Conclusion

Both optimization problems were successfully formulated, converted to a solvable standard form, and solved using manually implemented iterative algorithms in Python (NumPy only, no external LP/transportation solver libraries). The Big-M Simplex Method converged to the optimal solution $x_1 = 0.4$, $x_2 = 1.8$ with minimum $Z = 3.4$, correctly driving both artificial variables out of the basis. The transportation problem's VAM initial solution was verified by the MODI method to already be optimal, yielding a minimum transportation cost of 515. Together, these results demonstrate a complete, from-scratch computational treatment of constrained linear optimization and transportation optimization.